

Standard Web Development Technology Stack

Overview

This document defines the standard technology stack, software architecture, and development tools to be used for all web application and company website projects. The selected technologies prioritize scalability, maintainability, security, performance, SEO, and industry best practices.

Frontend

Languages

- HTML5
- CSS3
- TypeScript

Technologies

- Next.js
- React
- Tailwind CSS

Purpose

- Build responsive and SEO-friendly web applications.
 - Develop reusable and maintainable user interfaces.
 - Ensure fast loading speeds and excellent user experience.
-

Backend

Language

- TypeScript

Technologies

- Node.js

- NestJS

Purpose

- Develop scalable RESTful APIs.
 - Implement secure authentication and authorization.
 - Maintain a modular and enterprise-ready backend architecture.
-

Database

Database Management System

- PostgreSQL

Purpose

- Store and manage relational data.
 - Support complex business relationships and transactions.
 - Provide high performance and scalability.
-

Software Architecture

Backend Architecture

- Clean Architecture

Project Organization

- Feature-Based Modular Architecture

Each feature is organized into the following layers:

- Presentation Layer
- Application Layer
- Domain Layer
- Infrastructure Layer

This architecture promotes separation of concerns, maintainability, testability, and scalability.

Frontend Architecture

The frontend follows a Feature-Based Architecture where every feature manages its own:

- Components
 - Hooks
 - Services
 - Types
 - Validation
 - State (when required)
-

API Communication

- REST API
 - JSON data format
 - HTTPS
 - JWT Authentication
-

Development Tools

Version Control

- Git

Source Code Hosting

- GitHub

API Testing

- Postman

API Documentation

- Swagger (OpenAPI)

Code Quality

- ESLint

Code Formatting

- Prettier

Package Management

- npm

Containerization

- Docker
- Docker Compose

Environment Configuration

- `.env` files for development, testing, and production
-

Deployment (Recommended)

Frontend

- Vercel

Backend

- Docker container deployed to a cloud provider (such as Render, Railway, Azure, AWS, or DigitalOcean)

Database

- PostgreSQL (managed cloud service or self-hosted)
-

Development Principles

- Clean Code
- SOLID Principles
- DRY (Don't Repeat Yourself)
- KISS (Keep It Simple, Stupid)
- Separation of Concerns
- Component Reusability

- Feature-Based Development
 - Mobile-First Responsive Design
 - Secure by Design
 - SEO Optimization
 - Performance Optimization
 - Accessibility (WCAG) considerations
-

Standard Technology Stack

Category	Technology
Frontend Languages	HTML5, CSS3, TypeScript
Frontend Framework	Next.js
UI Library	React
Styling	Tailwind CSS
Backend Language	TypeScript
Backend Runtime	Node.js
Backend Framework	NestJS
Database	PostgreSQL
API Style	REST API
Authentication	JWT
Backend Architecture	Clean Architecture
Frontend Architecture	Feature-Based Architecture
Project Organization	Feature-Based Modular Architecture
Version Control	Git
Repository Hosting	GitHub
API Testing	Postman
API Documentation	Swagger (OpenAPI)
Code Quality	ESLint
Code Formatting	Prettier
Package Manager	npm
Containerization	Docker & Docker Compose
Frontend Deployment	Vercel
Backend Deployment	Docker on Render, Railway, Azure, AWS, or DigitalOcean

Goal

This stack serves as the standard foundation for developing secure, scalable, maintainable, SEO-friendly, and responsive web applications. It is intended to be reused across all future client projects to ensure consistency, code quality, and efficient development.